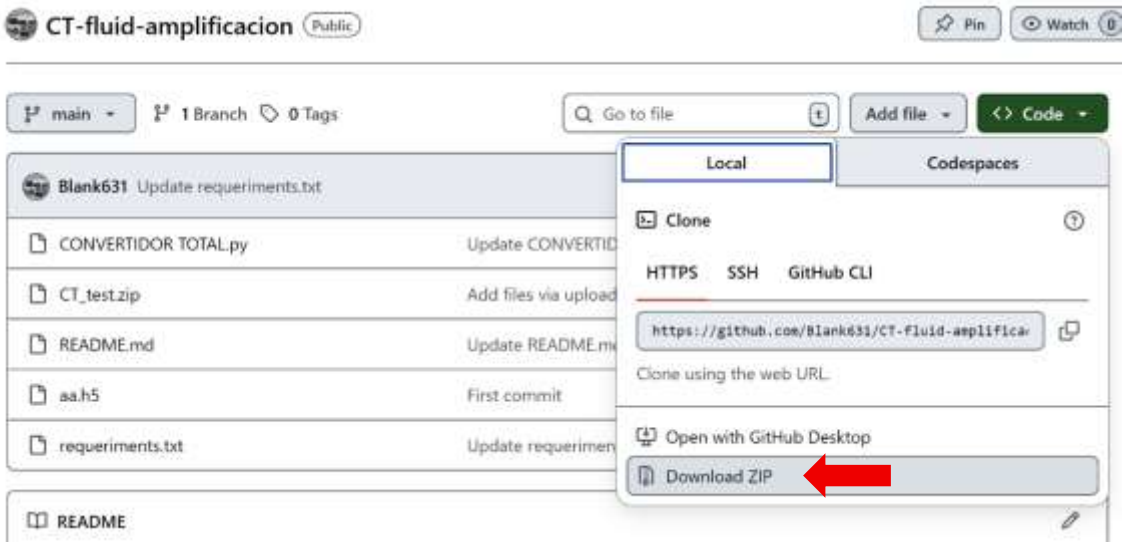
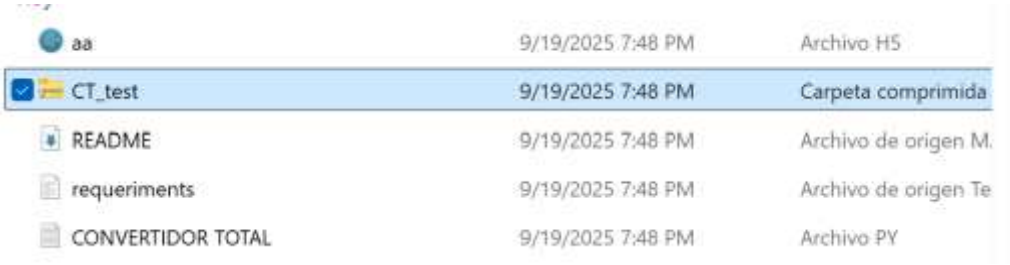
INSTRUCTIONS

Before starting, read the requirements. It is necessary to have a Python environment installed and, at the end, the 3D Slicer software for evaluation.

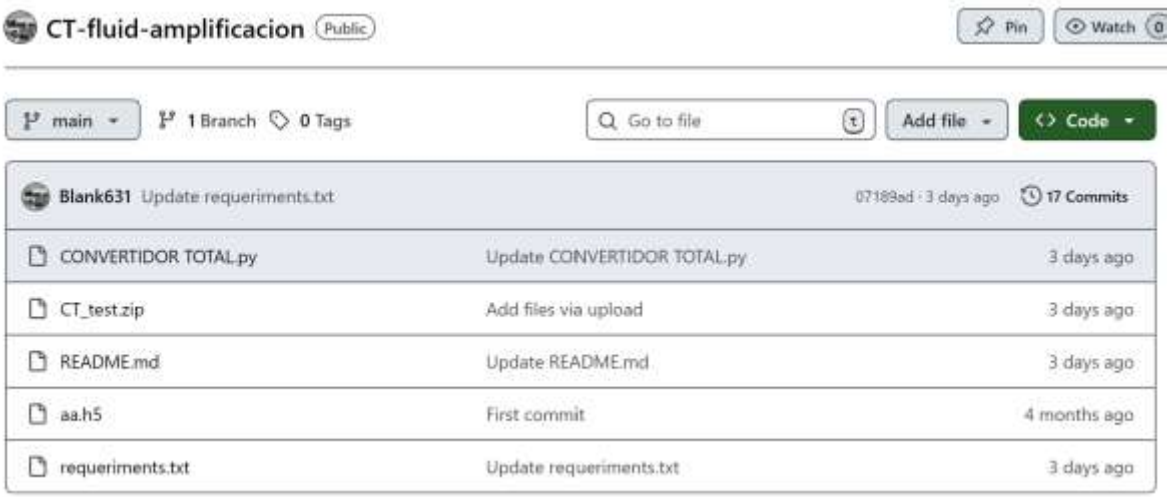
1. Download the folder and extract it.



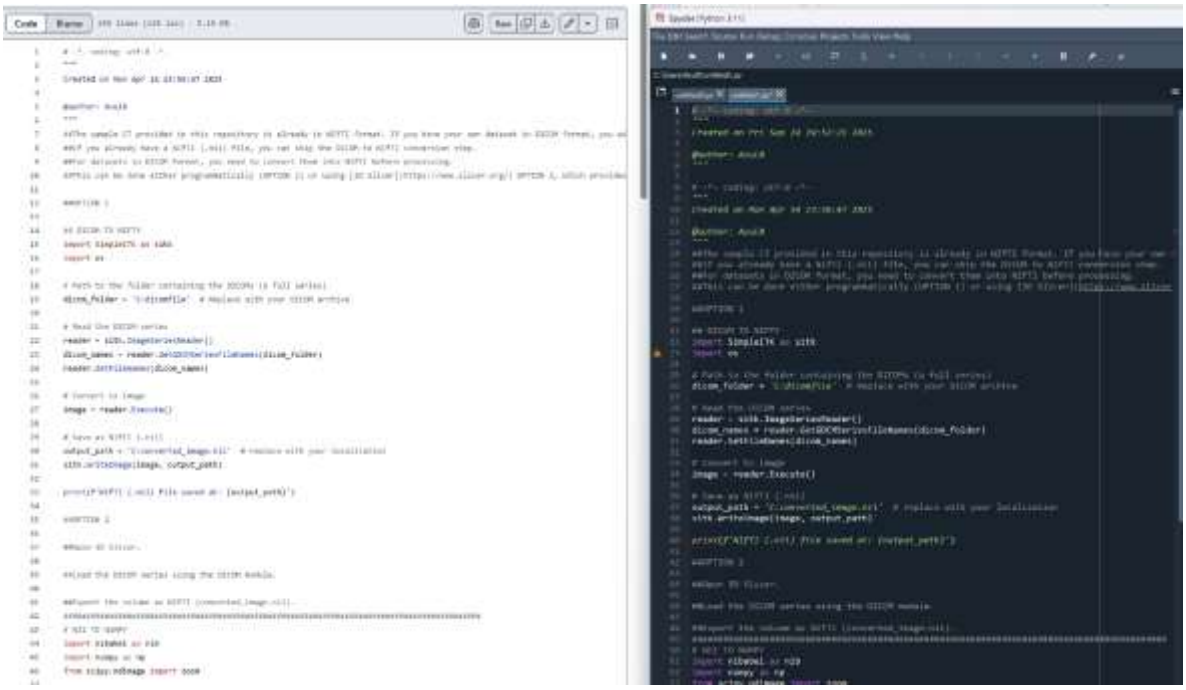
2. If you plan to use the test images, unzip **CT_test.zip** and extract the file **convertido.nii**.



3. Open the **CONVERTIDOR TOTAL** file on GitHub.



4. Copy the Python code into your preferred IDE; in this case, we will use **Spyder**.



5. If you are going to use the test images, skip ahead to the notes under **#NII TO NUMPY**, add the path to the **.nii** file and the location where you want to save the new file (**np.save**), then run the script. If not, first convert the DICOM files to NIfTI.

```python
# NII TO NUMPY
import nibabel as nib
import numpy as np
from scipy.ndimage import zoom

# Path to the image
TC_path = 'C:/Users/Azul8/Downloads/CT-fluid-amplificacion-main/CT-fluid-amplificacion-main/CT_test\convertido.nii'

def load_volume(path):
    """
    Load an image volume from a NIfTI file.
    """
    nii = nib.load(path)
    volume = nii.get_fdata()
    return volume


def resize_volume(volume, new_shape=(128, 128, 128)):
    """
    Resize a volume to new_shape.
    You can adjust this function to preserve aspect ratio or use different interpolation.
    """
    # Compute zoom factors
    zh, zw, zd = np.array(new_shape) / np.array(volume.shape)
    # Apply zoom (simplified, consider adjusting interpolation as needed)
    resized_volume = zoom(volume, (zh, zw, zd), order=1)  # order=1 (bilinear) is generally sufficient
    return resized_volume

# Load volumes
image_volume = load_volume(TC_path)

# Resize volumes
image_resized = resize_volume(image_volume)

# Expand dimensions to meet model input expectations (add a channel axis at the end)
image_resized = np.expand_dims(image_resized, axis=-1)

np.save('C://Users//Azul8//OneDrive//Escritorio//unet imagenes//archivos numpy//TC.npy', image_resized)
```

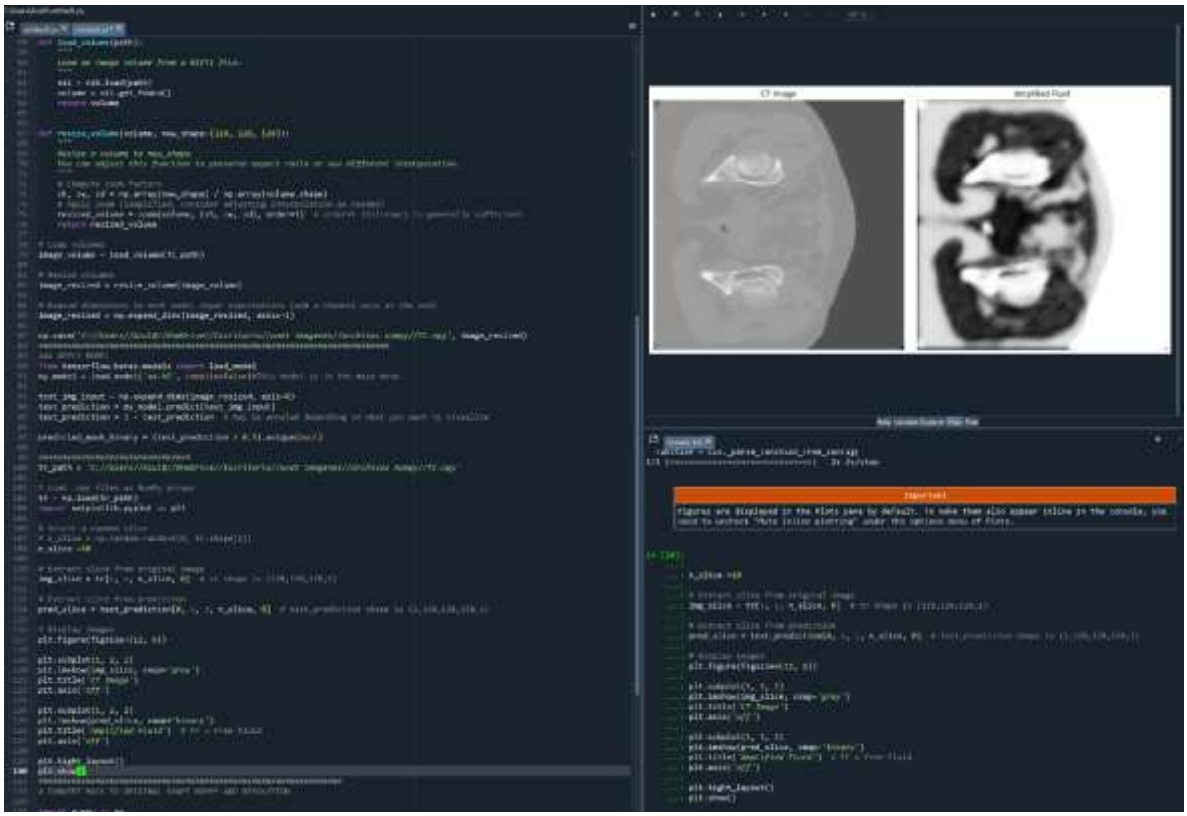
6. You now have your file in NumPy format ready to be fed into the neural network. Run the model; the **aa.h5** file is available in the repository.

```python
### APPLY MODEL
from tensorflow.keras.models import load_model
my_model = load_model('aa.h5', compile=False)#This model is in the main menu

test_img_input = np.expand_dims(image_resized, axis=0)
test_prediction = my_model.predict(test_img_input)
test_prediction = 1 - test_prediction  # may be annuled depending on what you want to visualize

predicted_mask_binary = (test_prediction > 0.5).astype(bool)
```

7. Apply it to the file and visualize the results.

```
tr_path = 'C://Users//Azul8//OneDrive//Escritorio//unet imagenes//archivos numpy//TC.npy'

# Load .npy files as NumPy arrays
tr = np.load(tr_path)
import matplotlib.pyplot as plt

# Select a random slice
# n_slice = np.random.randint(0, tr.shape[2])
n_slice =30

# Extract slice from original image
img_slice = tr[:, :, n_slice, 0]  # tr shape is (128,128,128,1)

# Extract slice from prediction
pred_slice = test_prediction[0, :, :, n_slice, 0]  # test_prediction shape is (1,128,128,128,1)

# Display images
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.imshow(img_slice, cmap='gray')
plt.title('CT Image')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(pred_slice, cmap='binary')
plt.title('Amplified Fluid')  # FF = Free Fluid
plt.axis('off')

plt.tight_layout()
plt.show()
```

This is what you should observe in the plots.



You now have the image. You can continue evaluating it in Python; if you need to assess it across all three axes (axial, sagittal, and coronal), you can convert it into a .nii file for use with software designed for medical image visualization.

8. To convert it into a .nii file and visualize it in medical imaging software, upload the original file (**convertido.nii**) so that its resolution can be copied and applied to the **TC.npy** image.

```python
# CONVERT .NPY BACK TO NIFTI WITH ORIGINAL SHAPE AND RESOLUTION

import numpy as np
import nibabel as nib
from scipy.ndimage import zoom
from tensorflow.keras.models import load_model

# --- 1) Rutas ---
ref_nii_path = r'C:\Users\AzuL8\Downloads\CT-fluid-amplificacion-main\CT-fluid-amplificacion-main\CT_test\convertido.nii'
npy_path     = r'C:\Users\AzuL8\OneDrive\Escritorio\unet imagenes\archivos numpy\7C.npy'
model_path   = r'ao.h5'
out_path     = r'C:\Users\AzuL8\OneDrive\Escritorio\unet imagenes\archivos nifty\amplified_fluid_prob.nii'

# --- 2) Cargar NIfTI de referencia (para affine/espaciado) ---
ref = nib.load(ref_nii_path)
ref_shape  = np.array(ref.shape)
ref_affine = ref.affine
ref_header = ref.header

# --- 3) Cargar volumen preprocesado (128x128x128x1) y predecir ---
vol128 = np.load(npy_path)                      # (128,128,128,1)
if vol128.ndim == 4 and vol128.shape[-1] == 1:
    vol128 = np.squeeze(vol128, axis=-1)        # (128,128,128)

model = load_model(model_path, compile=False)
pred = model.predict(np.expand_dims(vol128, axis=(0, -1)))  # -> (1,128,128,128,1)
pred = np.squeeze(pred, axis=(0, -1))           # (128,128,128)

# Opcional: invertir tal como en tu script
pred = 1.0 - pred

# --- 4) Reescalar al tamaño del NIfTI original (para overlay 1:1 en Slicer) ---
factors = ref_shape / np.array(pred.shape, dtype=float)
pred_resampled = zoom(pred, factors, order=1)  # lineal para mapa continuo

# --- 5) Normalizar (0..1) para que se vea "intensidad alta = fluido" ---
pmin, pmax = float(pred_resampled.min()), float(pred_resampled.max())
if pmax > pmin:
    pred_norm = (pred_resampled - pmin) / (pmax - pmin)
else:
    pred_norm = np.zeros_like(pred_resampled, dtype=np.float32)
pred_norm = pred_norm.astype(np.float32)

# --- 6) Guardar como NIfTI con geometría del estudio ---
nii = nib.Nifti1Image(pred_norm, affine=ref_affine, header=ref_header)
nii.set_sform(ref_affine, code=1)
nii.set_qform(ref_affine, code=1)
nib.save(nii, out_path)

print(f"✅ Guardado: {out_path}")
```

9. Now we can visualize the **amplified_fluid_prob.nii** file for analysis!